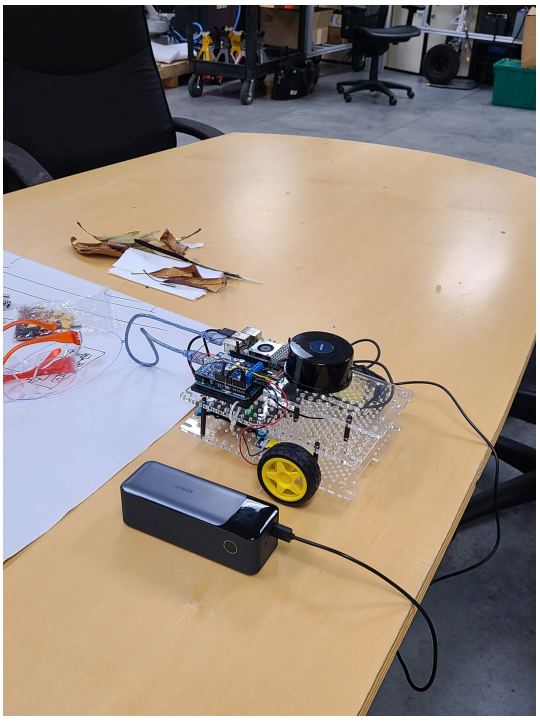
ROS 2 JAZZY / MOBILE ROBOTICS / 2026

Encoderless AGV Localization and SLAM

Laser odometry + inertial fusion on Raspberry Pi 5



Physical AGV prototype. Mapping was tested by manually moving the platform.

The engineering objective

Build a practical 2D mapping stack before wheel encoders and powered actuation are available. Consecutive LiDAR scans supply motion estimates; an IMU supports heading estimation; an EKF and SLAM Toolbox integrate the system.

My implementation work

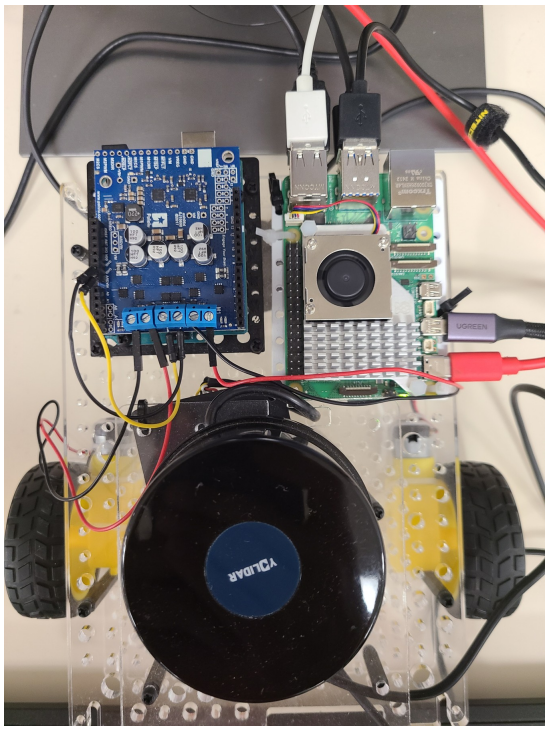
Developed the Arduino-to-ROS IMU interface and integrated the sensor drivers, launch files, estimator configuration, transforms, visualization, and covariance-testing workflow. Used existing RF2O, robot_localization, SLAM Toolbox, and YDLIDAR packages.

Python / C++ | ROS 2 | RF2O | EKF | SLAM Toolbox | RViz

dakshithaama1994@gmail.com | LinkedIn profile

01 / PHYSICAL INTEGRATION

A portable sensing platform





Top view of the platform and bench integration of compute, LiDAR, and microcontroller hardware. Images supplied by the author.

Raspberry Pi 5 + portable power

Runs ROS 2 Jazzy and the mapping stack; supplied from an Anker power bank. A USB hub supports keyboard and mouse peripherals.

YDLIDAR TG30

USB-connected planar laser scanner. The configured scan range is 0.10-10 m; these settings define processing limits rather than a demonstrated accuracy specification.

Arduino Uno + BNO055

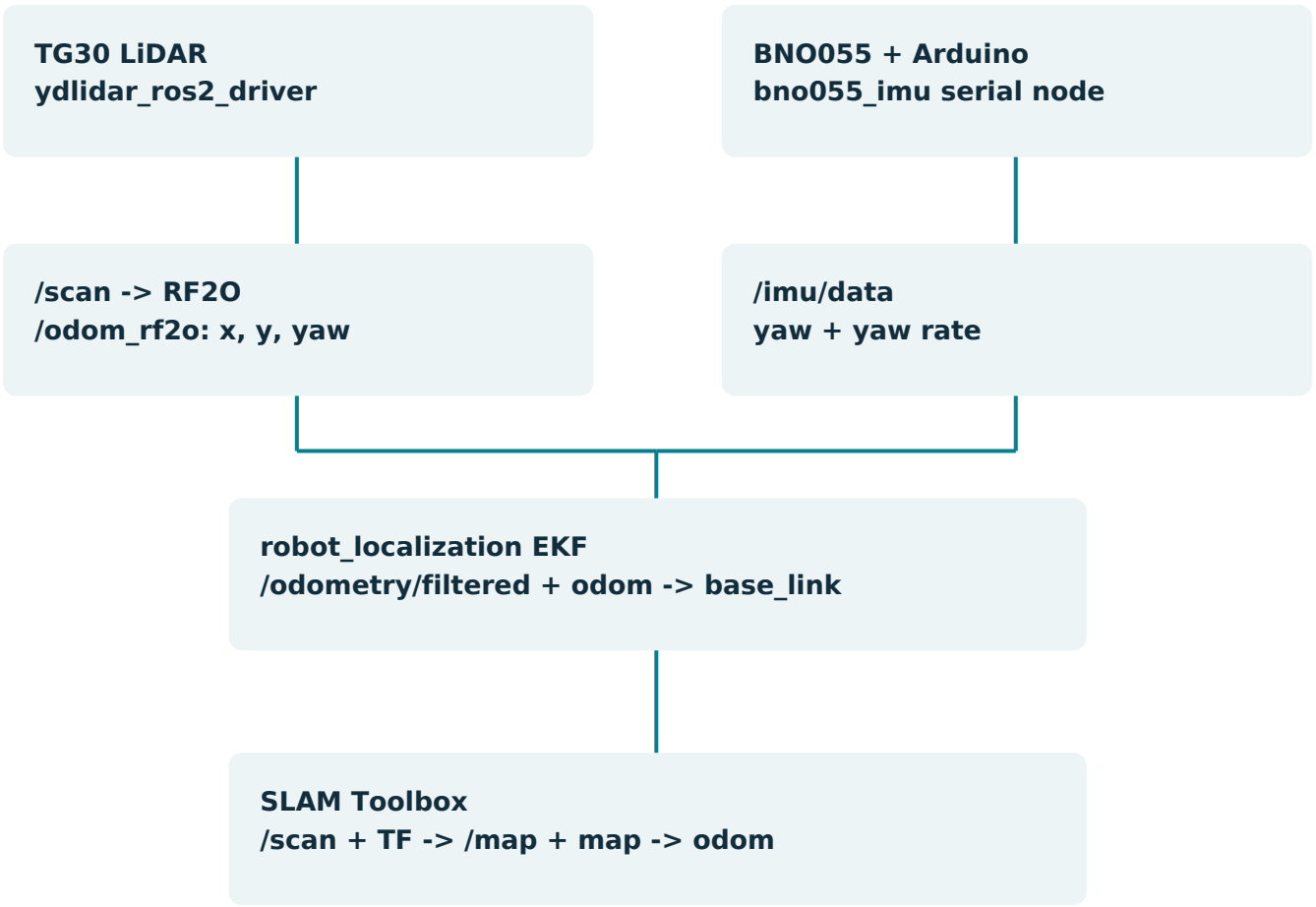
I2C sensor acquisition and serial transmission to the Pi. Quaternion, angular rate, acceleration, magnetic field, temperature, and calibration data are exposed to ROS.

Actuation boundary

The Pololu Dual G2 driver and two brushed DC motors are present but inactive in the documented mapping tests. Powered driving, motor feedback, and closed-loop navigation are future integration work.

02 / DATA FLOW & FRAMES

One integrated mapping pipeline



The scan stream also feeds SLAM Toolbox directly. The filtered motion estimate reaches SLAM through TF; this is not a direct odometry-topic subscription.

Transform ownership

SLAM Toolbox publishes map -> odom. The EKF publishes odom -> base_link. RF2O TF publication is disabled to avoid duplicate ownership; static transforms connect the LiDAR and IMU frames.

map -> odom -> base_link -> laser_frame / imu_link

Integration entry point: ros2 launch agv_localization agv.launch.py
The master launch includes sensors, RF2O, static TF, EKF, SLAM, and optional RViz.

03 / CUSTOM EMBEDDED INTERFACE

A custom IMU-to-ROS interface

A structured serial protocol

The Arduino sends a 27-field CSV packet with an IMU header, sequence number, Arduino time, quaternion, gyroscope, raw and gravity-removed acceleration, gravity vector, magnetic field, temperature, and calibration states.

Defensive parsing and standard outputs

The Python node validates the header and field count, handles parsing and serial faults, and retries connections. It timestamps publications with ROS time and converts magnetic readings from microtesla to tesla.

/imu/data	sensor_msgs/Imu
/imu/mag	sensor_msgs/MagneticField
/imu/temperature	sensor_msgs/Temperature
/imu/calibration	std_msgs/UInt8MultiArray

Stationary dataset evidence

The included rosbag metadata records 287.23 seconds and 8,610 /imu/data messages: approximately 30 messages/s averaged over the recording. The 100 Hz code setting is nominal, not a verified delivered rate.

Covariance choices

Published initial diagonal values: orientation [0.01, 0.01, 0.02]; angular velocity [0.02, 0.02, 0.02]; linear acceleration [0.04, 0.04, 0.04]. These are conservative tuning values, not the raw stationary variances recorded in code comments.

Quaternion-component variance is not Euler-angle covariance. Motors-off stationary data also cannot characterize powered vibration, magnetic disturbance, or motion-dependent errors. Repeat controlled recordings before final tuning.

04 / EKF & SLAM CONFIGURATION

Sensor fusion and mapping settings

Configuration verified from the submitted YAML and launch files

EKF update setting	15 Hz; two_d_mode enabled; world_frame: odom
RF2O input	/odom_rf2o; fuse x, y, yaw; differential: false
IMU input	/imu/data; fuse yaw and yaw rate; relative: true
Acceleration	Published by the IMU node, but disabled in EKF fusion
SLAM mode	Asynchronous mapping; scan matching and loop closure enabled
Map resolution	0.05 m/cell; grid spacing, not localization accuracy
Motion thresholds	0.10 m travel and 0.10 rad heading change
Map update setting	2.0 s; scan processing limits 0.10-10 m

Why these decisions matter

Encoderless translation

RF2O supplies relative motion from successive laser scans. IMU heading and yaw rate complement that estimate without requiring wheel encoders.

Relative IMU heading

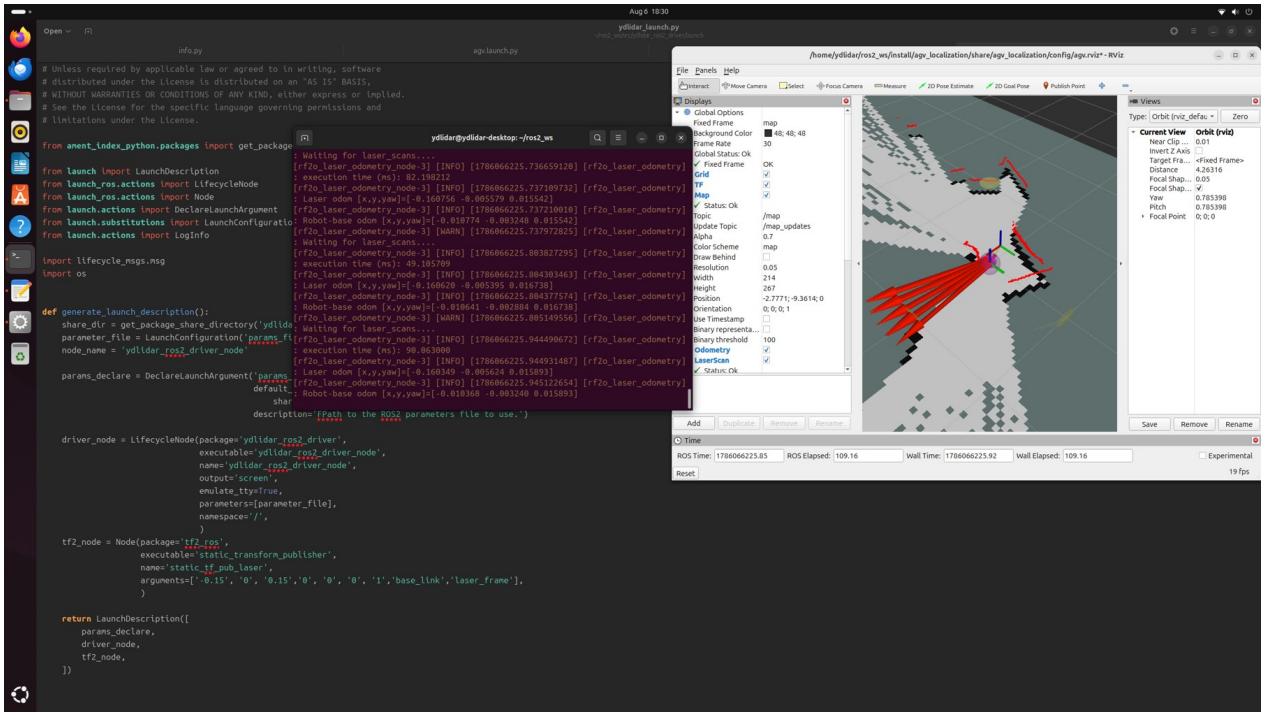
The initial IMU orientation becomes the heading reference. This supports local fusion while still requiring correct sensor axes, mounting transforms, and magnetic-environment validation.

Avoid unvalidated acceleration integration

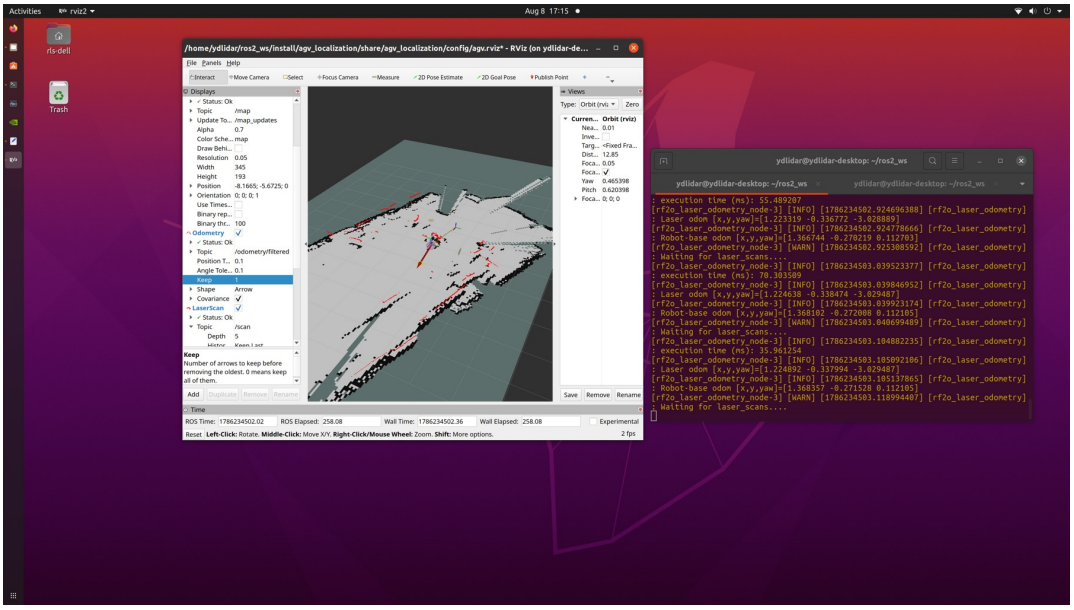
Keeping acceleration fusion disabled avoids adding an unvalidated drift source during early mapping tests. Controlled comparisons are needed before enabling additional states.

05 / DOCUMENTED OUTPUTS

Mapping during manual movement



Development screenshot: RViz shows an occupancy grid, laser returns, and robot visualization alongside active ROS processes.



Later mapping session: occupancy-grid structure with scan overlays. These are separate supplied views, not a controlled before/after comparison.

Evidence scope: screenshots demonstrate visual mapping output. No ground-truth trajectory, map-error score, repeatability study, or measured loop-closure improvement was supplied.

06 / ENGINEERING ASSESSMENT

Toward autonomous navigation

Demonstrated outcome

Integrated a launchable encoderless mapping prototype with a custom IMU interface, laser odometry, 2D sensor fusion, and occupancy-grid visualization. Manual movement allowed mapping development before drivetrain commissioning.

What the project shows

ROS 2 package integration; sensor-message design; serial protocol handling; covariance interpretation; estimator configuration; transform ownership; RViz diagnostics; and embedded/software interfacing.

Current limitations

Motor control and wheel feedback are not implemented in the documented phase. Static sensor offsets remain provisional. RF2O depends on scan quality and scene geometry; BNO055 yaw can be affected by magnetic conditions. Quantitative accuracy is not established.

Next validation steps

Measure LiDAR/IMU extrinsics; repeat IMU recordings across warm-up and powered-motor conditions; compare RF2O-only, yaw-rate-only, and yaw-plus-yaw-rate fusion; evaluate drift, map consistency, and recovery under turns.

Path toward navigation

Commission a protected motor supply and command interface, add wheel encoders, validate local odometry, then integrate Nav2 with robot geometry, velocity limits, obstacle layers, and emergency-stop behavior.

SOURCE MATERIAL & ATTRIBUTION

Prepared from the supplied Project Description.pdf, Arduino sketch, custom IMU node, EKF/SLAM YAML, launch files, stationary rosbag metadata, covariance tool, hardware photos, and RViz screenshots. RF2O, robot_localization, SLAM Toolbox, YDLIDAR, and Adafruit libraries are existing dependencies; contributions here center on interfacing, configuration, and integration.

Review method: source and document inspection; no ROS stack or physical hardware execution was performed for this portfolio.